

Optimization & Uncertainty in Learning Report FAQ

CS7641: Machine Learning

Summer 2026

1 Frequently Asked Questions

This FAQ is intentionally extensive. Everything required for the assignment is already stated in the project description, but each term many of the same questions come up as students begin reusing their SL backbone, implementing optimizer ablations, counting compute, running randomized optimization, and writing the report.

You do not need to absorb this entire document at once. There is a lot of information at the beginning of the assignment, and that can feel like a lot before you have started working. Use this FAQ as a guide while you move through the experimentation and writing process. When you are checking your SL baseline, read the SL connection section. When you are implementing RO, read the RO section. When you are building figures, read the figures and REPRO section. When you are writing the Discussion, read the discussion and writing section.

The OL Report is open-ended in analysis, but the expectations are concrete. The central idea is simple: you are not just running optimizers. You are building a controlled optimization study that explains how training interventions affect one PyTorch neural network on one multiclass classification problem.

2 Project Scope

Q: What exactly is the task?

You must perform multiclass classification using the Forest Cover Type dataset, also known as Covertype. The target is `Cover_Type`, and the classes are 1 through 7. You should use the approximately 20,000-instance Covertype sample from your SL Report unless a change is explicitly justified.

The OL Report uses the PyTorch neural network you finalized in the SL Report as the fixed backbone. The point is to explain how optimization and regularization interventions affect validation loss, test metrics, class-level behavior, stability, and compute cost.

You should not introduce a second dataset, collapse the target into a binary task, treat the target as regression, or replace Covertype with a different classification problem.

Q: What are the main parts of the assignment?

The assignment has three required experimental parts and one optional extra credit part. Part 1 applies Randomized Hill Climbing, Simulated Annealing, and a Genetic Algorithm to the final 1–3 layers of the PyTorch network. Part 2 runs Adam-family optimizer ablations on the full network. Part 3 studies regularization using standard Adam with the best Part 2 hyperparameters. Part 4 is optional extra credit and integrates the best Adam settings, best regularization combination, and best RO fine-tuning method under constrained additional search.

Q: Is this a new supervised learning assignment?

No. The SL Report asked which supervised learning methods worked well on Covertype. The OL Report asks why your PyTorch neural network trains the way it does and which training interventions matter under controlled budgets.

Do not redo the full SL assignment. Do not compare DT, kNN, SVM, and NN again. This assignment focuses on training interventions applied to the PyTorch network from SL.

3 Connection to the SL Report

Q: What carries over from the SL Report?

You should reuse the same Covertypes sample, target definition, train/validation/test split, preprocessing strategy, PyTorch neural-network backbone, and held-out test set from SL. This continuity is what makes OL interpretable. If the dataset, split, preprocessing, or architecture changes, then differences in results may be caused by those changes rather than by the optimizer or regularizer.

Q: What is my OL baseline?

Your OL baseline is the **PyTorch SGD-only neural network from your SL Report**. It is not necessarily the best model from your SL Report.

For example, if your best SL model was an SVM or Decision Tree, that model is not the OL backbone. OL is about the PyTorch neural network training process, so comparisons should be made against the SL PyTorch SGD-only model.

Q: What if my SL PyTorch model was not the best model?

That is fine. The OL Report does not require the PyTorch model to be the strongest SL model. The point is to study how training interventions affect the same PyTorch backbone.

A weaker SL PyTorch baseline can still produce a strong OL Report if you clearly explain whether optimizer changes, regularization, or RO fine-tuning improved training behavior, generalization, stability, or compute efficiency.

Q: What if I found a bug in my SL PyTorch model or preprocessing?

You may correct a real bug, but you must disclose the correction clearly. State what changed, why it was necessary, and how the change affects comparison to the SL baseline.

Do not silently change the architecture, split, preprocessing, or target mapping and then present the comparison as if it were identical to SL.

Q: Can I improve the architecture before starting OL?

No, not as a new architecture search. The assignment is designed around a fixed backbone so that the experimental changes are attributable to optimization and regularization.

If there is a genuine implementation error in the SL model, disclose and fix it. If the change is merely an attempt to make the model stronger, it undermines the controlled comparison.

Q: Can I use my scikit-learn MLP from SL?

No. For the OL Report, use only the PyTorch neural network from the SL Report. The optimizer ablations, custom update rules, regularization modules, seed control, and epoch-level diagnostics require PyTorch-level control.

4 Data, Metrics, and Baseline

Q: What metrics are required?

You should report validation loss, test Accuracy, test Macro-F1, test balanced accuracy, a confusion matrix, and per-class precision, recall, and F1 discussion.

Validation loss is central because many optimizer comparisons are about training dynamics and speed-to-target. Macro-F1 and balanced accuracy are central because Covertypes is multiclass and imbalanced. Accuracy alone is not enough because it may hide weak performance on minority cover types.

Q: Why do I need both validation loss and classification metrics?

Validation loss and classification metrics answer different questions.

Validation loss helps diagnose optimization and probability behavior. A method may reduce cross-entropy without changing many final class predictions. Macro-F1, balanced accuracy, and per-class metrics show whether those changes actually improve classification behavior across classes.

A strong report discusses cases where these metrics disagree. For example, Adam may lower validation loss quickly but produce little improvement in Macro-F1. Regularization may slightly reduce Accuracy but improve minority-class recall. These are meaningful outcomes if explained.

Q: Do I need to redo EDA?

You do not need a full new EDA section like the SL Report. You should briefly confirm the Covertypes facts that matter for OL: the sample size, target classes, class distribution, feature preprocessing, split sizes, and whether anything changed from SL.

If nothing changed, state that clearly. If something changed, disclose and justify it.

Q: What if my class distribution or sample changed?

A material change to the sample or class distribution affects comparability to SL. If the sample changed, explain why, report the new class distribution, and mark the comparison to SL as approximate.

A new sample is not automatically forbidden, but it weakens the causal story unless there is a strong reason.

5 Hypotheses

Q: What hypothesis do I need to include?

You need at least one clear hypothesis about expected optimization or regularization behavior before running the full experiments.

A strong hypothesis predicts a behavior or trade-off, not just a winner. It should be grounded in course concepts such as momentum, adaptive scaling, bias correction, regularization, seed stability, or compute cost.

Q: What does a strong OL hypothesis look like?

A strong hypothesis might say that Adam-family optimizers should reach a validation-loss threshold faster than SGD because adaptive second-moment scaling should reduce sensitivity to feature and gradient scale, but that this speed advantage may not guarantee higher Macro-F1 if the main errors are concentrated in minority cover types.

Another strong hypothesis might predict that regularization will reduce the train-validation gap and improve minority-class behavior more than RO fine-tuning because regularization changes the full training process, while final-layer RO can only make local adjustments after the backbone representation has already been learned.

A third strong hypothesis might predict that RO fine-tuning will provide limited gains relative to its function-evaluation cost because the final-layer parameter space remains high-dimensional and validation-loss improvements may not change hard class predictions.

Q: What does a weak hypothesis look like?

Weak hypotheses are too generic or retrospective. Statements like “Adam will perform best,” “regularization will help,” or “GA will be good because it is powerful” do not explain the expected mechanism, metric, trade-off, or evidence needed to test the claim.

Q: Do I need to revisit the hypothesis?

Yes. Later in the report, you must accept, reject, or qualify your hypothesis using quantitative evidence.

Use at least two kinds of evidence, such as validation loss, test Macro-F1, balanced accuracy, class-level behavior, seed stability, gradient evaluations, function evaluations, or wall-clock time. Do not leave the hypothesis hanging.

6 Compute, Seeds, and Fair Comparisons

Q: What does “budget-matched” mean?

Budget-matched means that methods are compared under fair and clearly reported compute limits. The exact budget type depends on the method.

For gradient-based methods, report gradient evaluations, optimizer updates, epochs, batch size, and wall-clock time. For RO methods, report function evaluations and wall-clock time. Do not compare one method trained much longer than another and then claim the difference is caused by the optimizer.

Q: What is a gradient evaluation?

For minibatch neural-network training, one minibatch backward pass can be counted as one gradient evaluation or optimizer update. If you use full-batch training, gradient accumulation, or an unusual training loop, define your counting rule clearly.

Epochs alone are not enough because two runs with the same epoch count may have different batch sizes, update counts, or wall-clock costs.

Q: What is a function evaluation?

For randomized optimization, every candidate objective computation counts as one function evaluation. If your validation objective is computed over the full validation set, that full validation pass counts as one function evaluation. If you use a fixed validation subset for computational reasons, one sweep over that fixed subset counts as one function evaluation.

For GA, compare by function evaluations, not just generations. A GA generation with population size 50 may require roughly 50 objective calls, so generation count alone is misleading.

Q: Do I need multiple seeds?

Yes, for main claims. The default expectation is at least 3 seeds unless computationally justified.

Seed reporting helps distinguish stable effects from lucky runs. If only one seed is used for a particular expensive diagnostic, explain why and avoid making strong stability claims from that run.

Q: What should I do with divergent or failed runs?

Report them. Do not silently remove them.

Divergence, plateauing, or failure to reach a validation-loss threshold is useful evidence about stability and sensitivity. If a method fails, mark it as failed or $>$ budget rather than deleting it from the comparison.

Q: What must stay fixed across comparisons?

Keep the dataset sample, split, preprocessing, PyTorch backbone, metric definitions, held-out test set, seed protocol, and initialization strategy fixed unless a change is necessary and justified. Keep hardware class and batch size fixed where possible, especially for gradient-based comparisons.

A fair comparison does not always require identical hyperparameters. SGD and Adam may need different learning-rate ranges. Fairness means that the search protocol, budgets, and evaluation rules are disclosed and comparable.

7 Part 1: Randomized Optimization

Q: What is the goal of Part 1?

Part 1 tests whether randomized optimization methods can usefully fine-tune the final layers of your PyTorch Coverttype network. The goal is not to prove that RO beats Adam. The goal is to evaluate whether RO provides useful tail tuning, robustness, or insight relative to its compute cost.

A good Part 1 analysis asks what the search process is doing. Is it finding quick local improvements and then plateauing? Is it exploring too broadly to make progress? Is it improving validation loss without changing hard class predictions? These are the kinds of questions that make Part 1 useful even if RO does not produce the best final score.

Q: Which RO algorithms are required?

You must run Randomized Hill Climbing, Simulated Annealing, and a Genetic Algorithm. All three should optimize the same validation objective on the same final-layer optimization problem.

The algorithms are meant to behave differently. RHC is a greedy local search with possible restarts. SA is a local search that can sometimes accept worse moves to escape a basin. GA is a population-based search that explores multiple candidates at once. Your analysis should use these differences to explain what you observe.

Q: What might RHC be doing?

Randomized Hill Climbing proposes a perturbation to the current weights and keeps it if the validation objective improves. This means RHC often improves quickly when there are easy local gains, then stalls when the current point is surrounded by worse nearby candidates.

If RHC is flat, do not immediately conclude that RHC is useless. Look at the perturbation scale, restart policy, and parameter count. A step size that is too small may produce tiny changes that cannot escape a plateau. A step size that is too large may jump over useful local improvements. No restarts may leave the search stuck in one basin.

A strong RHC discussion connects the best-so-far curve to search behavior. Fast early improvement followed by a long plateau suggests local exploitation. Repeated improvements after restarts suggest that the starting point matters. No improvement across restarts suggests that the perturbation design, parameter scope, or function-evaluation budget may be limiting the method.

Q: What might SA be doing?

Simulated Annealing is similar to local search, but it can accept worse candidates with a probability controlled by temperature. Early in the run, higher temperature can allow exploration. Later in the run, lower temperature should make the search more selective.

If SA behaves almost exactly like RHC, the temperature may be too low or the cooling schedule may be too aggressive. If SA wanders without improving, the temperature may be too high, the step size may be too large, or the cooling schedule may be too slow. If SA improves after an early period of instability, that may indicate it escaped a local basin and later exploited a better region.

A strong SA discussion does not just report the final score. It explains whether the temperature schedule produced exploration, exploitation, or noise.

Q: What might GA be doing?

A Genetic Algorithm maintains a population of candidate weight vectors and uses selection, crossover, mutation, and possibly elitism to search. GA can explore more broadly than RHC or SA, but it can also burn function evaluations quickly because every population member must be evaluated.

If GA improves best-so-far validation loss but requires many more function evaluations, the result may not be compute-efficient. If GA improves early and then stagnates, the population may have lost diversity. If GA is highly unstable across seeds, the mutation rate, population size, or selection pressure may be too aggressive. If the best candidate disappears between generations, the elitism policy may be missing or weak.

A strong GA discussion compares progress by function evaluations, not generations. A generation is not a fair unit of comparison unless population size is also considered.

Q: Why only the final 1–3 layers?

RO over all neural-network parameters is usually too expensive and high-dimensional for this assignment. Freezing earlier layers keeps the search space smaller and makes function-evaluation comparisons more realistic. You must report which layers are unfrozen and the exact number of trainable RO parameters.

Q: What if my final layers exceed 50,000 trainable parameters?

Reduce the number of trainable layers or choose a smaller final-layer scope that preserves the SL backbone as much as possible. Explain the choice and report the trainable parameter count.

Do not silently run RO over a much larger parameter space and then claim that RO failed.

Q: What objective should RO optimize?

RO should optimize validation loss. Evaluation should be deterministic. Use `model.eval()` so dropout is off and any normalization layers behave in evaluation mode.

Do not use the test set as the RO objective.

Q: Can I use a validation subset for RO?

Yes, if full validation evaluation is computationally prohibitive. The subset must be fixed, disclosed, seeded, and separate from the test set. Stratification is recommended when possible.

Final test metrics should still be evaluated on the held-out test set.

Q: What if RO improves validation loss but not Macro-F1?

That is a very plausible result. Validation loss can improve when predicted probabilities become better calibrated or more confident, even if the final predicted class labels barely change. Macro-F1 changes only when class predictions change in ways that affect per-class precision and recall.

This is especially important for Covertypes because minority classes may be difficult to move. If RO mostly improves confidence on majority-class examples, validation loss can improve while Macro-F1 or balanced accuracy stays flat.

A good report explains the disagreement rather than treating it as a contradiction.

Q: What do I need to disclose for RHC, SA, and GA?

For RHC, report the restart policy, perturbation distribution, perturbation scale, step-size schedule, plateau rule if used, and stopping rule. For SA, report the initial temperature, cooling schedule, perturbation scale, acceptance rule, and stopping rule. For GA, report the population size, selection method, crossover operator, mutation rate, mutation distribution, elitism policy, number of generations, and stopping rule.

The details matter because these are not minor implementation choices. They often determine whether the method explores usefully, stalls, or burns compute without improving the objective.

Q: Should RO beat Adam or the SL baseline?

Not necessarily. RO may produce small improvements, no improvements, or improvements that are not worth the compute. A strong report can conclude that RO was not useful if the comparison is fair and the explanation is clear.

Q: What are the biggest RO pitfalls?

The most common RO problems are non-deterministic evaluation, unequal budgets, missing operator disclosures, comparing GA by generations instead of function evaluations, and hiding failed runs.

8 Part 2: Adam-Family Ablations

Q: What is the goal of Part 2?

Part 2 studies which optimizer ingredients affect training speed, stability, and generalization on the full PyTorch Covertypes backbone.

This is not a tour of optimizers. It is an ablation study. You should explain what changed when momentum, Nesterov acceleration, adaptive scaling, bias correction, and decoupled weight decay were introduced or removed.

Q: Which optimizer variants are required?

You must run SGD with no momentum, SGD with momentum, Nesterov momentum, Adam, Adam without bias correction, Adam with $\beta_1 = 0$ as an RMSProp-like condition, and AdamW with decoupled weight decay.

Q: What might plain SGD be doing?

Plain SGD uses the current minibatch gradient to update parameters without momentum or adaptive scaling. It can be slow, noisy, and sensitive to the learning rate, but it is also a useful baseline because it shows what happens without modern optimizer ingredients.

If SGD improves slowly but steadily, that may indicate the model can learn but needs many updates. If SGD diverges, the learning rate may be too high. If SGD stagnates, the learning rate may be too low, the loss surface may be poorly conditioned, or the model may need momentum or adaptive scaling.

A strong SGD discussion treats it as a reference point for understanding what momentum and Adam-family methods change.

Q: What might momentum be doing?

Momentum accumulates a moving direction of past gradients. It can smooth noisy minibatch updates and accelerate learning when gradients point in a consistent direction. On Covertypes, this may reduce jagged trajectories or help SGD reach a validation-loss threshold faster.

Momentum can also overshoot if the learning rate is too high. If SGD with momentum oscillates or diverges while plain SGD is stable, the combination of learning rate and momentum may be too aggressive.

A good report asks whether momentum improved speed, stability, or both.

Q: What might Nesterov momentum be doing?

Nesterov momentum uses a lookahead-style update. In some settings, it can improve convergence behavior relative to classical momentum. In other settings, it may behave very similarly to standard momentum, especially if the learning rate and momentum values are conservative.

If Nesterov and momentum are nearly identical, that is not automatically a problem. It may suggest that the model, data, or chosen hyperparameter range is not especially sensitive to the difference between these two momentum variants.

Q: What might Adam be doing?

Adam combines first-moment smoothing with second-moment adaptive scaling. It often reduces sensitivity to gradient scale and can reach lower validation loss faster than SGD, especially early in training.

Faster validation-loss reduction does not automatically mean better test Macro-F1 or balanced accuracy. Adam may improve confidence or loss without correcting minority-class errors. If Adam is fast but does not improve class-level behavior, that is an important result.

A strong Adam discussion distinguishes speed-to-threshold, final validation loss, train-validation gap, seed stability, and final test metrics.

Q: What might Adam without bias correction be doing?

Adam’s bias correction matters most early in training because first- and second-moment estimates are initially biased toward zero. Removing bias correction can change early update magnitudes and may slow, destabilize, or alter early training behavior.

If the no-bias-correction condition differs only early and later catches up, that still supports a meaningful conclusion: bias correction mattered most for early-phase optimization rather than final performance.

Q: What might Adam with $\beta_1 = 0$ be doing?

Setting $\beta_1 = 0$ removes Adam’s first-moment smoothing while keeping second-moment adaptive scaling. This condition is RMSProp-like in spirit because it keeps adaptive scaling but removes momentum-like averaging of the gradient direction.

If this condition is less stable than Adam, that may suggest first-moment smoothing mattered. If it performs similarly to Adam, that may suggest adaptive scaling mattered more than momentum for this backbone. If it is highly reactive or noisy, that may reflect the loss of gradient smoothing.

Q: What might AdamW be doing?

AdamW decouples weight decay from the adaptive gradient update. This can change generalization behavior relative to standard Adam, especially when weight norm control matters.

AdamW should not be confused with standard Adam or with coupled L2 regularization. If AdamW improves test Macro-F1 or balanced accuracy without the largest validation-loss improvement, that may indicate that decoupled weight decay improved generalization rather than pure optimization speed.

Q: Is AdamW the same as Adam with L2 weight decay?

No. AdamW uses decoupled weight decay. Standard Adam with coupled L2 regularization is not the same thing. Make sure the report clearly distinguishes Adam, Adam with any coupled weight decay, and AdamW.

Q: What does Adam without bias correction mean?

Adam uses bias correction to adjust early moment estimates. Removing bias correction changes early update behavior. This may require a custom optimizer or careful implementation.

If you implement this yourself, document the update rule and include inspectable code in the REPRO materials.

Q: Do I need the same learning rate for every optimizer?

Not necessarily. Different optimizer families often require different learning-rate ranges. However, the tuning strategy must be fair and disclosed. Do not give one optimizer an extensive search and another optimizer one arbitrary setting.

A reasonable approach is to use coarse, comparable searches over sensible ranges and then compare the selected settings under matched budgets.

Q: What is ℓ , and how should I choose it?

ℓ is a fixed validation-loss threshold used for speed-to-target analysis. Choose it using training/validation evidence only. A reasonable choice is a validation-loss value reached by at least one optimizer during preliminary runs, or a fixed improvement relative to the SL baseline.

Do not choose ℓ based on test performance. Do not change ℓ separately for each optimizer.

Q: What should the sensitivity heatmaps show?

The heatmaps should show how optimizer behavior changes over coarse grids such as (α, β_1) and (α, β_2) . They should help answer whether performance is robust across a broad region or sensitive to a narrow set of hyperparameters.

A broad stable region suggests the optimizer is forgiving. A narrow good region suggests the method can work but may be fragile. Divergence at high learning rates is not unusual and should be marked rather than hidden. Weak sensitivity to β_1 or β_2 may mean the learning rate dominates behavior or that the training budget is too short to expose the effect.

Q: What if the optimizer with the best validation loss is not best on Macro-F1?

That is plausible and worth discussing. Validation loss measures probability quality, while Macro-F1 reflects discrete class predictions across classes. A method can reduce loss by becoming more confident on already-correct majority-class predictions without improving rare-class recall.

In that case, your conclusion should not simply say which optimizer “won.” It should explain which optimizer was best for speed, which was best for validation loss, which was best for test classification metrics, and which was most stable.

Q: Do all heatmaps need to fit in the main report?

No. The main report should include compact heatmap summaries or the most informative heatmap. Full heatmaps may go in the REPRO PDF.

The main report still needs to interpret the heatmap behavior. Do not move all interpretation to REPRO.

Q: What are common Part 2 mistakes?

Common mistakes include changing the model architecture, using different splits, changing batch size without explanation, hiding divergent runs, using AdamW as the Adam baseline, comparing different compute budgets, and reporting only the best seed.

9 Part 3: Regularization Study

Q: What is the goal of Part 3?

Part 3 measures how regularization affects generalization under fixed optimizer settings. It asks whether regularization changes validation loss, train-validation gaps, test Macro-F1, balanced accuracy, minority-class behavior, and seed stability.

The key question is not only whether the score went up. The stronger question is what kind of behavior changed. Did the regularizer reduce overfitting? Did it improve rare-class recall? Did it make training more stable across seeds? Did it improve validation loss but hurt Accuracy? These are the kinds of interpretations that make Part 3 useful.

Q: Why must standard Adam stay fixed?

Part 3 is about regularization, not a second optimizer sweep. Use standard Adam with the best Part 2 hyperparameters. Do not switch to AdamW. Do not retune the learning rate or betas inside Part 3 unless a deviation is necessary and explicitly justified.

Keeping Adam fixed lets you attribute changes to regularization rather than to optimizer retuning.

Q: What regularizers are required?

You must evaluate L2 weight decay, early stopping, dropout, label smoothing, and modest input noise or augmentation applied only during training. You must report standard Adam with no added regularization, the best single regularizer, and the best regularization combination.

Q: What might L2 weight decay be doing?

L2 weight decay penalizes large weights and can reduce overfitting by discouraging overly complex parameter solutions. If L2 helps, you may see a smaller train-validation gap, improved validation loss, or better test Macro-F1. If L2 is too strong, training and validation performance may both decline because the model is underfitting.

A good L2 discussion asks whether weight decay improved generalization or simply constrained the model too much.

Q: What might early stopping be doing?

Early stopping prevents training from continuing after validation performance stops improving. It can act like a regularizer because it stops the model before it overfits the training data.

If early stopping helps, the training curve may show validation loss flattening or worsening while training loss continues improving. If early stopping hurts, the patience may be too short or the validation signal may be noisy.

A good early-stopping discussion reports what was monitored, the patience value, the maximum budget, and whether best weights were restored.

Q: What might dropout be doing?

Dropout injects noise into hidden representations during training. It can reduce co-adaptation among hidden units and improve generalization, but it can also hurt compact tabular models if it removes too much useful signal.

If dropout helps, you may see reduced train-validation gap or improved test metrics. If dropout hurts, it may be over-regularizing the network or disrupting useful feature interactions. Dropout effects may also differ across classes, especially if minority-class patterns are already hard to learn.

Q: What might label smoothing be doing?

Label smoothing softens the target labels during training. It can reduce overconfidence and sometimes improve validation loss or calibration-like behavior. However, it may not change hard class predictions enough to improve Macro-F1.

If label smoothing lowers validation loss but leaves Macro-F1 unchanged, that is still interpretable. It may have improved probability behavior without changing class decisions. If it hurts minority classes, the smoothing may be reducing the model's ability to separate rare classes sharply.

Q: What might input noise be doing?

Input noise acts as a data-level regularizer by making the model less sensitive to small feature perturbations. For scaled continuous features, modest noise may improve robustness. Too much noise can blur class boundaries and make similar cover types harder to separate.

Input noise should be applied only during training. If it is applied to validation or test data as augmentation, the evaluation no longer measures the original task cleanly.

Q: What is coupled L2 weight decay?

Coupled L2 weight decay means the penalty is part of the loss or gradient update in the standard optimizer setting. This differs from AdamW's decoupled weight decay. Part 3 uses standard Adam, so be clear about

how L2 is implemented.

Q: Is AdamW allowed in Part 3?

No. AdamW is part of the Part 2 optimizer ablation. Part 3 should use standard Adam with the best Part 2 hyperparameters and then vary regularization mechanisms.

Q: How should I use early stopping fairly?

Define the monitored quantity, patience, maximum budget, and whether best weights are restored. If early stopping uses fewer epochs than other conditions, say whether it saved compute or created a different training opportunity.

Do not tune early stopping on the test set.

Q: Where should dropout be placed?

Dropout placement should be documented clearly. State which layers receive dropout and what rates are used. Do not simply say “we used dropout.”

Dropout can help or hurt tabular neural networks. If it hurts performance, explain whether it likely over-regularized the model or disrupted useful feature interactions.

Q: How should I use label smoothing?

Label smoothing should be applied during training through the training loss. State the smoothing magnitude. Evaluate normally on validation and test data.

Label smoothing may improve loss stability or reduce overconfidence without improving Macro-F1. That is a valid result if explained.

Q: What counts as modest input noise?

Input noise should be small, training-only, and justified. It should not be applied to validation or test data as an augmentation. If features are scaled, choose noise magnitudes that make sense in the scaled feature space.

Excessive input noise can blur class boundaries and harm minority classes.

Q: Can regularization hurt performance?

Yes. Regularization can underfit, reduce minority-class recall, or lower Accuracy. That does not automatically mean the experiment failed. The important question is whether the report explains how the regularizer changed validation loss, train-validation gap, class-level behavior, and stability.

Q: How do I choose the best single regularizer and best combination?

Use validation evidence, not test performance. The test set should be used only for final evaluation.

A good report shows compact selection evidence in the main report and places detailed sweeps in REPRO if they are too large.

Q: What if the best combination is not the combination of every regularizer?

That is common. Regularizers can interact. Dropout plus strong weight decay plus label smoothing plus input noise may over-regularize the model. The best combination may be smaller and more targeted.

A strong report explains why the combination helped or hurt relative to the best single regularizer.

10 Extra Credit: Integrated Best Combination

Q: What is the extra credit asking for?

The extra credit asks whether the best pieces from Parts 1–3 work together. Use the best standard Adam hyperparameters from Part 2, the best regularization combination from Part 3, and the best RO fine-tuning method from Part 1 on the same final 1–3 layers.

Q: Is extra credit a new search?

No. It is a composition experiment, not a new broad search. Do not introduce new optimizers, change the backbone, change the RO layer scope, or run a fresh wide hyperparameter sweep.

Q: What should I report for extra credit?

Report whether the integrated recipe improves test metrics beyond the best result from Parts 1–3, whether the improvement is stable across seeds, and whether the additional compute is worth it.

11 Figures, Tables, and REPRO

Q: What figures and tables should be in the main report?

The main report should include compact versions of the most important evidence: the SL PyTorch baseline versus best OL interventions, validation loss versus compute, RO best-so-far objective versus function evaluations, Adam-family optimizer trajectories, sensitivity heatmap summaries, seed stability summaries, regularization comparisons, final metric tables, and class-level diagnostics.

The exact layout is up to you. A small number of well-designed multi-panel figures is usually stronger than many disconnected plots.

Q: What belongs in the REPRO PDF?

The REPRO PDF should include the material needed to verify and reproduce your work. This includes run commands, environment details, package versions, random seeds, data paths, sample/split/preprocessing/backbone confirmation, full heatmaps, full seed tables, detailed regularization sweeps, logs, and additional diagnostics.

Q: Can I put full heatmaps and detailed sweeps in REPRO?

Yes. That is encouraged when the main report is crowded.

The main report must still include compact summaries and interpretation. REPRO supports the paper; it does not replace the paper.

Q: Can all my interpretation go in REPRO?

No. The main report must stand alone. If a claim is central to your conclusion, the main report needs at least compact evidence and interpretation for that claim.

Q: How do I fit this into 8 pages?

Use compact, multi-panel figures and summary tables. Avoid repeated plots that do not change the argument.

A good strategy is to use one figure for RO progress, one compact figure or table for Adam-family results, one compact figure or table for regularization, one final summary table, and one class-level diagnostic. Extended supporting materials can go in REPRO.

12 Discussion and Writing

Q: What should the Discussion do?

The Discussion should synthesize the whole OL study. It should not simply summarize Part 1, then Part 2, then Part 3.

A strong Discussion explains which intervention primarily improved optimization speed, which intervention primarily improved generalization, which intervention improved stability, which intervention affected class-level behavior, whether validation-loss improvements transferred to test metrics, whether the improvement justified the compute cost, and what practical decision rule follows from the evidence.

Q: What is cross-part synthesis?

Cross-part synthesis means explaining what the parts reveal together.

Weak:

Adam was best. RO was slow. Regularization helped a little.

Stronger:

Adam reached the validation-loss threshold with fewer gradient evaluations than SGD, but the improvement did not translate into higher Macro-F1. Regularization had a smaller effect on validation loss but reduced the train-validation gap and improved minority-class recall, suggesting that the main bottleneck was not optimization speed alone. RO fine-tuning produced minor validation-loss gains but required many function evaluations, so it was not the best compute-value trade-off for this backbone.

The stronger version explains the type of improvement each intervention produced.

Q: How do I distinguish optimization, generalization, stability, and efficiency?

These are different claims.

Optimization is about training dynamics, such as faster validation-loss reduction or fewer updates to reach ℓ . Generalization is about held-out behavior, such as test Macro-F1, balanced accuracy, class-level recall, or train-validation gaps. Stability is about seed-to-seed variation and failure rates. Efficiency is about whether the improvement was worth the wall-clock time, gradient evaluations, or function evaluations.

Do not treat these as interchangeable.

Q: What does class-level analysis look like?

Class-level analysis looks beyond aggregate scores. Discuss which cover types improved, which remained difficult, and whether minority classes benefited or suffered.

Use per-class precision, recall, F1, or confusion-matrix patterns. A method that improves Accuracy while hurting minority-class recall should not be described as simply “better” without qualification.

Q: What should limitations look like?

Limitations should be concrete and tied to the study.

Good limitations include the approximately 20,000-instance sample, rare-class instability, final-layer-only RO scope, limited RO function-evaluation budget, coarse heatmaps, limited seed count, validation-subset use for RO, hardware constraints, or dependence on the SL PyTorch backbone architecture.

A generic statement like “more tuning could be done” is not enough unless you explain which result motivates more tuning and what that tuning would test.

Q: What is a practical decision rule?

A practical decision rule tells the reader what to do next time based on your evidence.

For example:

For this Coverttype backbone, Adam-family tuning should be prioritized first when the goal is speed-to-threshold, but regularization should be prioritized when the train-validation gap or minority-class recall is the main concern. RO fine-tuning should be used only as a final-layer diagnostic or tail-tuning step because its validation-loss gains were small relative to the function-evaluation cost.

That is much stronger than simply naming the best method.

Q: How do I avoid writing a checklist?

A checklist reports what you did. A technical report explains why the result matters.

Weak:

I ran RHC, SA, GA, Adam, AdamW, dropout, and early stopping. AdamW had the best score.

Stronger:

The optimizer ablations show that adaptive scaling reduced the number of updates needed to reach the validation-loss threshold, but the regularization study shows that improved speed alone did not resolve the train-validation gap. The best regularized Adam condition produced a smaller validation-loss improvement than AdamW but improved balanced accuracy, suggesting that generalization and class-level behavior were more sensitive to regularization than optimizer speed.

Use the stronger style.

13 Sources, AI Use, and Academic Integrity

Q: Do I need outside sources?

Yes. You need at least two peer-reviewed references beyond course materials. These sources should be used to justify or interpret choices related to optimizers, Adam/AdamW, regularization, randomized optimization, imbalanced multiclass metrics, leakage controls, or stability analysis.

Primary sources are strongly preferred. Textbooks should be avoided for the required sources. arXiv papers, preprint servers, blog posts, documentation pages, tutorials, lecture notes, Wikipedia, Medium posts, and vendor documentation do not count as the required peer-reviewed sources.

Use IEEE citation style. Cite sources in the body of the report where they support a claim. Do not only list sources in the bibliography.

Q: Can I use documentation or tutorials?

Yes, for implementation help. Documentation and tutorials can help you use PyTorch, scikit-learn, or plotting tools. They do not satisfy the required peer-reviewed source requirement.

Q: Can I use AI tools?

Yes, with disclosure. You may use AI tools for brainstorming, outlining, grammar edits, code generation, code refactoring, and debugging. You may not submit AI-written analysis, conclusions, or figures as your own.

Every submission must include an AI Use Statement that identifies which tools you used, what they helped with, and that you reviewed and understood all assisted content.

Q: What is the easiest way to get into academic integrity trouble?

Submitting analysis, code, or figures that are not your own without proper disclosure or citation. This includes copying from classmates, public repositories, websites, or AI tools.

The report must reflect your own experiments and your own interpretation.

14 Submission and Access

Q: Do I need to use the IEEE conference template?

Yes. The main report must use the official IEEE conference paper template, such as:

```
\documentclass[conference]{IEEEtran}
```

A generic article template or custom report format is not acceptable for the main report.

Q: Do I need to use Georgia Tech / Enterprise Overleaf?

Yes. The main report must be created using your Georgia Tech / Enterprise Overleaf account.

You must provide a read-only Overleaf link. Do not send email invitations.

Q: Do I need to use GT Enterprise GitHub?

Yes. Course-related code must be stored in GT Enterprise GitHub. Your reproducibility sheet must include a single commit hash from the final version of your code.

Personal GitHub repositories are not acceptable for this assignment.

Q: What do I submit?

You submit two PDFs: **OL_Report_{GTusername}.pdf**, which is the formal IEEE-style report, and **REPRO_OL_{GTusername}.pdf**, which is the reproducibility sheet.

Q: What must be in the REPRO PDF?

The REPRO PDF must include the read-only Overleaf link, Git commit hash, run commands, environment details, package versions, data paths, random seeds, sample/split/preprocessing/backbone confirmation, compute accounting, and extended supporting materials that do not fit in the main report.

15 Work Planning

Q: How should I chunk the work?

A practical workflow is to separate the project into phases rather than trying to run everything at once.

First, verify that the SL PyTorch baseline can be reproduced. Confirm the sample, split, preprocessing, metrics, and test baseline. Next, build a shared training and evaluation harness that handles seeds, metrics, validation loss, test evaluation, runtime, gradient evaluations, and output logging.

Then run small pilot experiments for RO so you understand parameter counts, perturbation scales, and function-evaluation costs before launching longer runs. After that, run Adam-family ablations with a coarse but fair protocol. Save optimizer trajectories, threshold results, and seed summaries. Then run the regularization study using standard Adam with the chosen Part 2 settings.

Finally, build compact figures for the main report, move full heatmaps and detailed sweeps to REPRO, and write the Discussion after you can compare across all three required parts.

Q: What should I write first?

Write the parts that do not depend on final results first: dataset continuity from SL, baseline description, hypothesis, split protocol, compute accounting, and experimental design.

Do not wait until every run is finished to start writing. Waiting usually leads to a report that lists results instead of explaining them.

16 Quick Self-Check

Before submitting, use this table as a final scan. This is not a substitute for the assignment description, but it should help you catch the most common missing pieces.

Area	Self-check question
Dataset and task	Did I use Covertypes and predict <code>Cover_Type</code> classes 1 through 7?
SL continuity	Did I reuse the SL sample, split, preprocessing, and PyTorch backbone, or clearly justify any changes?
Baseline	Did I compare against the SL PyTorch SGD-only baseline, not just the best SL model overall?
Hypothesis	Did I state a hypothesis before the full experiments and revisit it with quantitative evidence?
Metrics	Did I report validation loss, Accuracy, Macro-F1, balanced accuracy, and class-level behavior?
Splits and leakage	Did I tune only on training/validation data and reserve the test set for final evaluation?
Compute accounting	Did I count and report gradient evaluations, function evaluations, wall-clock time, seeds, and hardware details?
Randomized Optimization	Did I run RHC, SA, and GA on the final 1–3 layers and report parameter count, operator settings, and progress curves?
Adam ablations	Did I complete all required Adam-family optimizer variants and interpret speed, stability, sensitivity, and generalization?
Regularization	Did I use standard Adam from Part 2 and evaluate L2, early stopping, dropout, label smoothing, and input noise or augmentation?
Discussion	Did I synthesize across Parts 1–3 rather than summarize them separately?
Class-level behavior	Did I discuss per-class behavior, minority-class performance, or confusion patterns?
Limitations	Did I include concrete limitations tied to my actual results and study design?
Decision rule	Did I end with a practical recommendation about when to prioritize optimizer changes, regularization, or RO fine-tuning?
Format	Did I use the IEEE conference template for the main report?
Access	Did I use Georgia Tech / Enterprise Overleaf and GT Enterprise GitHub?
Sources	Did I cite at least two peer-reviewed sources in IEEE style?
AI Use Statement	Did I include a clear AI Use Statement?
REPRO PDF	Did I submit a REPRO PDF with commands, seeds, environment details, compute accounting, and extended supporting materials?

17 Version Control

- v1.0 – 05/19/2026 – TJL refactored OL Report FAQ for Summer 2026 Covertypes assignment.

FAQ document written by Theodore LaGrow.